Introduction

Every developer writes this bug at least once, and almost nobody notices it the first time. You write a `Dog` class. It works. You write a `Cat` class right after, and because it's basically the same shape, you copy `Dog`, rename a few things, and move on. It feels efficient. It is efficient — for about four classes.

Then you add a `Bird`. Then a `Fish`. Then a `Snake`. Now you have five classes that all repeat the same `__init__` method, the same attribute setup, the same boilerplate. And here's the part that actually costs you time: the day you need to change how an animal stores its name, you don't make one edit. You make five, in five different files, and if you miss even one, you've introduced a bug that won't show up in testing — it'll show up in production, three weeks from now, on the one class nobody remembered to update.

This is exactly the problem inheritance was designed to solve, and it's also one of the most commonly tested concepts in coding interviews — not because interviewers love object-oriented theory, but because how you handle shared structure across related classes tells them a lot about how you'll handle a real codebase. Understanding inheritance, method overriding, `super()`, and duck typing isn't academic. It's the difference between a codebase that scales cleanly to twenty classes and one that becomes unmaintainable at five.

Problem Overview

Here's the situation in plain terms: you have several classes that represent different kinds of the same underlying concept — different animals, different shapes, different payment methods, it doesn't matter. Each of these classes needs some shared setup (like storing a name) and some behavior that's unique to each one (like how it makes a sound).

The naive approach is to write each class independently, copying the shared parts. The correct approach is to identify what's actually shared, pull it into one common class, and let every specific class build on top of that common class instead of duplicating it. On top of that, each class still needs a way to say "I'm mostly like the others, except for this one behavior" — and Python needs a way to treat all of these different classes uniformly when you just want to call a shared action on any of them, regardless of their exact type.

Example

Let's say you're modeling animals in a small game or simulation. You want:

- Every animal to have a `name`.
- Every animal to have its own `make_sound()` behavior.
- A way to loop over a list of different animals and just call `make_sound()` on each, without caring what specific type each one is.

```
animals = [Dog("Rex"), Cat("Whiskers"), Bird("Tweety")]

for animal in animals:
    print(animal.make_sound())
```

Expected output:

```
Rex says Woof!
Whiskers says Meow!
Tweety says Tweet!
```

Notice what's happening here: the loop doesn't know or care that it's holding a `Dog`, a `Cat`, and a `Bird`. It just calls `make_sound()` on whatever object it has, and each object responds in its own way. That behavior — same method call, different result depending on the object — is the entire payoff of this lesson.

Intuition

Here's how an experienced engineer actually arrives at this design, step by step, without jumping straight to code.

Step one: notice duplication. If you're writing the same `__init__` logic in `Dog`, `Cat`, `Bird`, `Fish`, and `Snake`, that's a signal, not a coincidence. Five near-identical blocks of code mean there's a shared concept hiding underneath all of them — in this case, "a thing that has a name."

Step two: name the shared concept. Give it its own class. Call it `Animal`. This class holds only what's truly common — the name, the basic setup — and nothing that varies.

Step three: connect the specific classes to the shared one. `Dog`, `Cat`, and `Bird` don't redefine the name logic. They declare that they *are* animals, and they automatically get everything `Animal` already knows how to do. This is inheritance: `class Dog(Animal):` reads as "Dog is a kind of Animal."

Step four: handle the parts that differ. Not every animal makes the same sound, so each subclass defines its own version of `make_sound()`. Python always uses the most specific version it can find for

a given object — this is called overriding, and it's what lets `Dog` and `Cat` share structure while still behaving differently where it matters.

Step five: handle the parts that differ *and* build on the shared part. If `Dog` needs extra setup — say, a `breed` — you don't want to retype the name-storing logic. Instead, you call the parent's setup first with `super().__init__(...)`, then add your extra piece. You're extending, not replacing.

Step six: realize you don't always need any of this. If all you actually need is "call this method on whatever object you're given," Python doesn't require that object to inherit from anything. It just checks whether the method exists. That's duck typing, and it's worth knowing precisely because it tells you when *not* to reach for inheritance.

Brute Force Solution

Idea: Write each class fully independently. `Dog`, `Cat`, `Bird`, `Fish`, `Snake` each define their own `__init__` and their own `make_sound`, with no shared parent.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return f"{self.name} says Woof!"


class Cat:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return f"{self.name} says Meow!"
```

Algorithm: Copy the class template, rename the class, change the return string in `make_sound`, repeat for every animal.

Advantages: - Extremely simple to read for a single class in isolation. - No need to understand inheritance at all.

Disadvantages: - Every shared change (e.g., validating that `name` isn't empty) must be made in every class. - Easy to introduce silent inconsistencies — one class quietly drifts from the others. - Doesn't scale. Five classes is annoying; twenty is unmanageable.

Time complexity: O(1) per method call — this isn't an algorithmic problem, it's a maintenance problem, and that's exactly why it matters. **Space complexity:** O(n) in duplicated code, where n is the number of classes — the real cost here isn't runtime memory, it's engineering time.

Optimal Solution

Pull the shared logic into a single parent class, and let every specific class inherit from it.

```
          Animal
         /   |   \
      Dog   Cat   Bird
```

`Animal` defines the trunk — the `__init__` that stores `name`. `Dog`, `Cat`, and `Bird` are branches. They automatically carry whatever `Animal` defines, without rewriting it. Each branch only needs to define what makes it different: its own `make_sound`.

Class	Defines `__init__`?	Defines `make_sound`?	Gets `name` from
Animal	Yes	No (base default)	itself
Dog	No	Yes	Animal
Cat	No	Yes	Animal
Bird	No	Yes	Animal

When `Dog` needs something extra, like a `breed`, it doesn't rewrite the name-storing logic — it calls `super().__init__(name)` to run the parent's setup first, then adds its own piece on top.

Python Code

```
class Animal:
    """Shared base for anything that has a name and can make a sound."""

    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return f"{self.name} makes a sound."


class Dog(Animal):
    def __init__(self, name, breed=None):
        super().__init__(name)  # reuse the parent's setup
        self.breed = breed

    def make_sound(self):
```

```
        return f"{self.name} says Woof!"


class Cat(Animal):
    def make_sound(self):
        return f"{self.name} says Meow!"


class Bird(Animal):
    def make_sound(self):
        return f"{self.name} says Tweet!"


class Person:
    """Not related to Animal at all — duck typing in action."""

    def __init__(self, name):
        self.name = name

    def make_sound(self):
        return f"{self.name} says Quack! (I'm just pretending)"


def make_it_sound(thing):
    """Works on anything with a make_sound method, regardless of ancestry."""
    return thing.make_sound()
```

Code Walkthrough

- `class Animal:` defines the shared base. It holds the `__init__` logic every animal needs.
- `def __init__(self, name): self.name = name` stores the name once, in exactly one place.
- `class Dog(Animal):` — the parenthesis is what makes `Dog` a subclass of `Animal`. It inherits everything `Animal` defines.
- `super().__init__(name)` inside `Dog.__init__` calls `Animal`'s setup first, so `Dog` doesn't need to retype the name-storing logic. It then adds `self.breed`, which is unique to `Dog`.
- `Cat` and `Bird` don't define `__init__` at all — they inherit it directly from `Animal`.
- Each subclass overrides `make_sound()` with its own version. Python always resolves to the closest, most specific definition for the object's actual type.
- `Person` inherits from nothing related to `Animal`, but still defines `make_sound()`.
- `make_it_sound(thing)` doesn't check `thing`'s type at all — it just calls `thing.make_sound()`. This works identically whether `thing` is a `Dog` or a `Person`, because Python only cares whether the method exists.

Dry Run

```
animals = [Dog("Rex", breed="Labrador"), Cat("Whiskers"), Bird("Tweety")]

for animal in animals:
    print(make_it_sound(animal))

print(make_it_sound(Person("Alex")))
```

Walking through this: 1. `Dog("Rex", breed="Labrador")` calls `Dog.__init__`, which calls `super().__init__("Rex")`, setting `self.name = "Rex"` via `Animal`, then sets `self.breed = "Labrador"`. 2. `Cat("Whiskers")` has no `__init__` of its own, so Python walks up to `Animal.__init__` and sets `self.name = "Whiskers"`. 3. The loop calls `make_it_sound(animal)` for each object. Python looks up `make_sound` on each object's actual class — `Dog.make_sound`, `Cat.make_sound`, `Bird.make_sound` — not on `Animal`. 4. Output: `Rex says Woof!`, `Whiskers says Meow!`, `Tweety says Tweet!`. 5. `Person("Alex")` isn't an `Animal` at all. `make_it_sound` doesn't check that — it just calls `.make_sound()`, which succeeds because `Person` happens to define it. Output: `Alex says Quack! (I'm just pretending)`.

Complexity Analysis

- **Time:** O(1) per `make_sound()` call — Python's method resolution order looks up the method on the object's class, which is a dictionary lookup, not a linear scan.
- **Space:** O(1) additional space per object beyond its own attributes — inheritance doesn't duplicate the parent's code into each subclass; it's shared once on the class itself.
- These are correct because inheritance in Python is implemented as a lookup chain (the MRO), not as a copy operation — the parent class's code exists exactly once in memory, regardless of how many subclasses use it.

Alternative Solutions

An alternative to inheritance here is composition combined with duck typing: instead of a class hierarchy, each object just implements whatever methods are needed, with no shared parent at all. This works well when the "shared" behavior is thin or when classes don't actually share meaningful state — only behavior. Inheritance wins when there's real shared state and setup logic (like `name`) that would otherwise be duplicated. Duck typing wins when you don't need shared state at all, just a shared method signature — as shown by `Person` fitting into `make_it_sound` without inheriting from `Animal`.

Edge Cases

- **Empty or missing name:** `Animal("")` — the base class doesn't validate this, so an empty string silently becomes a valid name. Add validation in `Animal.__init__` if this matters.
- **Subclass forgets to override `make_sound`:** it silently falls back to `Animal.make_sound`, which may produce a generic, unintended message instead of an error.
- **Calling `super().__init__()` with the wrong arguments:** raises a `TypeError` immediately, which is actually a good thing — it fails loudly instead of silently.
- **An object with no `make_sound` method at all passed into `make_it_sound`:** raises `AttributeError` at call time, not before — duck typing has no compile-time safety net.
- **Deep inheritance chains (Animal → Mammal → Dog):** `super()` still resolves correctly, but debugging which class actually defines a given method gets harder as the chain grows.

Common Mistakes

1. Forgetting to call `super().__init__()` in a subclass that also needs the parent's setup, resulting in missing attributes like `name`.
2. Copy-pasting `__init__` into every subclass instead of inheriting it — the exact bug this lesson is built around.
3. Overriding a method and accidentally changing its signature, breaking any code that calls it polymorphically.
4. Reaching for inheritance when duck typing would be simpler — creating a rigid hierarchy for objects that only need to share one method, not shared state.
5. Assuming Python checks class types before calling a method — it doesn't, which means `isinstance` checks are sometimes unnecessary defensive code.

Interview Questions

- What's the difference between overriding a method and overloading it in Python?
- What does `super()` actually return, and why does it matter in multiple inheritance?
- How does Python decide which method to call when a class inherits from more than one parent?
- What is duck typing, and when would you choose it over inheritance?
- Can you explain polymorphism using this Animal example, without using the word "polymorphism"?

Similar Problems

- **Implement a Shape hierarchy (Circle, Square, Triangle) with an `area()` method** — same pattern: shared interface, different implementation per subclass.
- **Design a payment system (CreditCard, PayPal, Bank Transfer)** — tests the same shared-setup-plus-overridden-behavior structure in a more realistic domain.
- **Abstract base classes with `abc.ABC`** — a natural next step once you understand overriding, since it lets you *force* subclasses to implement a method.
- **Multiple inheritance / diamond problem** — directly foreshadowed at the end of this lesson, and a common follow-up interview question.

Key Takeaways

- If you're copy-pasting a class, you're probably missing a shared parent class.
- `class Child(Parent):` is the entire mechanism behind inheritance — Child gets everything Parent defines.
- Overriding lets a subclass replace a specific method while keeping everything else inherited.
- `super().__init__()` lets you extend a parent's setup instead of duplicating it.
- Duck typing means Python checks capability, not ancestry — sometimes that's all you need, and inheritance is overkill.

Watch the Video

This entire walkthrough — the bug, the fix, and the family tree diagram — is demonstrated live in the video. Watch it here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around one idea: real engineering intuition comes from tracing bugs and rebuilding the fix from scratch, not from memorizing syntax. Each episode picks a concept that quietly breaks codebases once they grow past a handful of files, and rebuilds it the way an experienced engineer actually would.

Call To Action

If this helped untangle inheritance for you, give the video a like on YouTube and subscribe so you don't miss the next one — we're covering multiple inheritance next. Subscribe to the Substack for the full written breakdown of every episode. Drop a comment if something felt off, unclear, or if there's a concept you want covered next. Share this with anyone still copy-pasting classes.

Quick Review

When 5th subclass repeats same setup code, what pattern fixes it?

Inheritance: pull shared state/behavior into a base class, subclasses inherit and extend it.

Why does copy-pasting logic across classes 'bite you later'?

One shared bug must now be fixed in every copy; miss one and behavior silently diverges.

What does calling super() in an overridden method actually do?

Runs the parent class's version first, so you extend behavior instead of fully replacing it.

Same method call, different result across subclasses — what's this called?

Polymorphism: each subclass's overridden method runs, chosen by the object's actual class at runtime.

Duck typing vs inheritance-based polymorphism — key difference?

Duck typing needs no shared base class at all; Python calls the method if the object simply has it.

Common override bug: forgetting super().__init__() — what breaks?

Parent's attributes/setup never run, so inherited methods silently fail on missing state.